When a new aggregator node is added to the cluster and the workers are spawned and registered, they start working, and the master's lag decreases:

```
[18:50:14,499] [DEBUG]--tick, current lag 351                                  ❶
[18:50:14,505] [DEBUG]--current count HashMap(this -> 16, a -> 32, very ->
➥16, simulates -> 16, simple -> 16, stream -> 32)                             ❷
[18:50:14,505] [DEBUG]--current count HashMap(this -> 17, a -> 34, very ->
➥17, simulates -> 17, simple -> 17, stream -> 34)
[18:50:14,505] [DEBUG]--current count HashMap(this -> 18, a -> 36, very ->
➥18, simulates -> 18, simple -> 18, stream -> 36)
[18:50:14,505] [DEBUG]--current count HashMap(this -> 19, a -> 38, very ->
➥19, simulates -> 19, simple -> 19, stream -> 38)
[18:50:14,505] [DEBUG]--current count HashMap(this -> 20, a -> 40, very ->
➥20, simulates -> 20, simple -> 20, stream -> 40)
[18:50:15,510] [DEBUG]--receiving, current lag 347
```

❶ Result when the master creates a new text

❷ Result when a text comes back aggregated from the worker

The lag accumulated in the master decreases more quickly than new texts are created. You would see a similar result if you dropped one of the two aggregators—the application would still work. This is the fault tolerance you expect from a cluster.

This simple case shows how Akka provides a powerful abstraction. You can build your business logic without worrying about degradation or failure and deal with it when you design the application entry point and roles. This essential part of Akka, along with supervision, provides the fault tolerance needed to build reliable distributed systems.

8.5.1 Serialization

Up to this point, sending messages between actors has not been a problem because they have run on the same JVM. But when actors